



Python / C++

▼ Python

▼ C++

기본 형태 (test.cpp)

```
#include <헤더파일 이름>

int main(){

    (... 세부 코드 ...)

    return 0; // 프로그램 정상 종료
}
```

test.cpp 실행 방식

```
g++ test.cpp -o test && ./test" // test라든 파일로 저장하고 test를 읽어오는 방식
```

예시 코드 (배터리 표시)

std::cout은 터미널 출력을 나타냄

```
#include <iostream>

int readBatterySensor() {
    // 실제로는 하드웨어 SDK나 ROS2 토픽 등을 통해 값을 가져옴.
    return 75;
}

int main() {
    int battery = readBatterySensor(); // 센서 데이터가 입력되는 지점
    std::cout << "Sensor Data: " << battery << "%" << std::endl;
    return 0;
}
```

```
g++ test.cpp -o test && ./test"
```

```
// [ 터미널 출력 형태 ]
Sensor Data: 75%
```

예시코드 (std::getline, std::cin - 입력)

```
#include <iostream> // input, output stream
#include <string>    // 문자열 사용하기 위함.
```

```

int main() {
    int ret = 1;
    // 0. 문자열 변수 선언 및 초기화 (보통 기본값은 "-"로 함. 빈칸 또는 공백은 인식 힘들.)
    std::string robot_description = "-";
    // 1. 타이틀 출력
    std::cout << "Enter Robot Desc: ";
    // 2. 설명 입력
    // std::cin >> robot_description; ← 원래 but 공백포함 저장하고 싶어 getline 사용.
    std::getline(std::cin, robot_description);
    // 3. 입력된 설명 출력
    std::cout << "Robot Desc: " << robot_description << std::endl;
    // 4. 반환값 변경
    ret = 0;
    // 5. 프로그램 종료
    return ret;
}

```

로봇 세부 내용 저장 코드

```

#include <iostream>
#include <string>

int main() {
    // 1. 로봇 이름을 저장하는 std::string 변수 만들기
    std::string robot_name = "Warehouse_bot";

    // 2. 로봇 ID를 저장하는 int 변수 만들기
    int robot_id = 12345;

    // 3. 최대 속도를 저장하는 double 변수 만들기
    double max_speed = 25.0; // m/s

    // 4. 자율주행 가능 여부를 저장하는 bool 변수 만들기
    bool autonomous = true;

    // 5. 로봇 등급을 저장하는 char 변수 만들기
    char grade = 'A';

    // 6. bool 값은 true/false로 출력하기 (미리 선언한 것.)
    std::cout << std::boolalpha;

    // 7. 모든 값을 보기 좋게 출력하기
    std::cout << "==== Robot Information =====" << std::endl;
    std::cout << "ROBOT NAME   : " << robot_name << std::endl;
    std::cout << "ROBOT ID     : " << robot_id << std::endl;
    std::cout << "Max Speed    : " << max_speed << " m/s" << std::endl;
    std::cout << "Autonomous   : " << autonomous << std::endl;
    std::cout << "Grade        : " << grade << std::endl;

    return 0;
}

```

결과

```

root@d1b87d26ec88:/workspace# g++ test2.cpp -o test3 && ./test3
===== Robot Information =====
ROBOT NAME : Warehouse_bot
ROBOT ID : 12345
Max Speed : 25 m/s
Autonomous : true
Grade : A

```

! for 문 실행 순서

```

for ( 1.초기화 ; 2.조건검사 ; 4.증감식 ) {
    3.본문 실행 (std::cout)
}

```

예시코드

```

#include <iostream>
#include <string>

int main(){
    for(int i=0; i<5; ){
        std::cout << "i++ : [" << i++ << "]" << std::endl;
    }

    std::cout << "======" << std::endl;

    for(int j=0; j<5; ){
        std::cout << "++j : [" << ++j << "]" << std::endl;
    }
    return 0;
}

// i++ , ++i 가 있음. ++이 앞에 있으면 더하기 부터 하는 것.
// 4.증감식에 i++,++j 넣어도 똑같이 출력됨. 출력하고 증감하기 때문.

```

출력값

```

root@2e14ec01f30d:/workspace# g++ for_test.cpp -o test1 && ./test1
i++ : [0]
i++ : [1]
i++ : [2]
i++ : [3]
i++ : [4]
=====
++j : [1]
++j : [2]
++j : [3]
++j : [4]
++j : [5]

```

▼ break

```

#include <iostream>

int main() {
    for (int step = 1; step <= 10; step++) {
        std::cout << "Step: " << step << std::endl;

        if (step == 5) {

```

```

        std::cout << "Emergency stop at step 5." << std::endl;
        break;
    }
}

return 0;
}

```

break 한 줄 요약 : "회전목마(Loop)에서 뛰어내리기"

- **탈출 대상**: 나를 감싸고 있는 "가장 가까운 반복문(for, while) 또는 switch 문".
- **if** 는 "언제 뛰어내릴지" 결정하는 스위치일 뿐, **break** 가 탈출시키는 것은 결국 밖의 반복문.
- **if** 문은 어차피 한 번 하고 끝나므로 탈출할 벽이 X → 단독 **if** 에서 **break** 쓰면 컴파일 에러.

📌 코드 한 줄 요약

```

for (...) {
    if (조건) break; // 🏃 for문 루프 즉시 탈출 (OK)
}

// ❌ 잘못된 사용: 단독 if문
if (조건) break; // ❌ 탈출할 루프 없음 (Error)

```

▼ ↺ continue

```

#include <iostream>

int main() {
    for (int sensor_id = 1; sensor_id <= 5; sensor_id++) {
        if (sensor_id == 3) {
            std::cout << "Sensor 3 skipped." << std::endl;
            continue;
        }

        std::cout << "Reading sensor " << sensor_id << std::endl;
    }

    return 0;
}

```

continue 한 줄 요약 : 이번 바퀴만 스킵 (다음 숫자로 점프).

sensor_id는 3일 때 continue로 빨려들어가고, 넘어가서 sensor_id++이 적용돼서 4가 되고 다시 for문으로 들어가서 조건 참이니까 4출력하고 5출력하고 끝나겠네. (3은 출력 X)

▼ 간단 예시 코드 (로봇 이동 / 배터리)

```

#include <iostream>

int main() {
    int battery = 100;
    bool obstacle_detected = false;

    //로봇이 최대 10단계 이동한다.
    for(int step=1; step<=10; step++){

```

```

std::cout << "step : " << step << std::endl;

//5단계에서 장애물이 감지된다.
if(step == 5){
    obstacle_detected = true;
}

//장애물이 감지되거나 배터리가 20 미만이면 정지한다.
if (obstacle_detected || battery < 20) {
    std::cout << "Robot stopped." << std::endl;

    if (obstacle_detected) {
        std::cout << "!!! obstacle detected !!!" << std::endl;
    }
    if (battery < 20) {
        std::cout << "!!! battery is lower than 20% !!!" << std::e
ndl;
    }
    break;
}

//각 단계마다 배터리가 10씩 감소한다.
battery -= 10;
std::cout << "BATTERY : [";
for (int j = 0; j < 10; j++) {
    if (j < 10-step) {
        std::cout << "■";
    } else {
        std::cout << " ";
    }
}
std::cout << "]" << battery << "%" << std::endl;

std::cout << "Robot moving forward." << std::endl;

for(int i=1; i<=step; i++){
    std::cout << " "; }
std::cout << ">>>>" << std::endl;
std::cout << "===== (OB)" << std::endl;
}

return 0;
}

```

결과

```

root@2e14ec01f30d:/workspace# g++ for_test.cpp -o test1 && ./test1
step : 1
BATTERY : [■■■■■■■■■■] 90%
Robot moving forward.
>>>▶
===== (OB)
step : 2
BATTERY : [■■■■■■■■■■] 80%
Robot moving forward.
>>>▶
===== (OB)
step : 3
BATTERY : [■■■■■■■■■■] 70%
Robot moving forward.
>>>▶
===== (OB)
step : 4
BATTERY : [■■■■■■■■■■] 60%
Robot moving forward.
>>>▶
===== (OB)
step : 5
Robot stopped.
!!! obstacle detected !!!

```

▼ Python / C++ 대표적인 문법 차이 모음

항목	C++ (규칙과 성능)	Python (간결과 편의)	한 줄 요약
시작점	<code>int main() { }</code>	(필요 없음)	C++은 입구가 정해져 있음
입력	<code>std::cin >> x;</code>	<code>x = input()</code>	받는 방향(>>) vs 넣는 방향(=)
출력	<code>std::cout << x;</code>	<code>print(x)</code>	보내기(<<) vs 출력하기(print)
마침표	<code>;</code> (세미콜론)	(없음)	C++은 문장 끝에 반드시 <code>;</code>
영역	<code>{ }</code> (중괄호)	<code> 들여쓰기</code>	C++은 가두고, Python은 민다
변수	<code>int x = 10;</code>	<code>x = 10</code>	C++은 이름표(타입)가 필수
논리	<code>true / false</code>	<code>True / False</code>	Python은 앞글자가 대문자
상수	<code>const int X = 1;</code>	<code>X = 1</code>	Python은 관례로만 '상수' 취급
함수	<code>void f() { }</code>	<code>def f():</code>	반환타입(<code>void</code>) vs 정의(<code>def</code>)
조건	<code>if (x > 0) { }</code>	<code>if x > 0:</code>	Python은 괄호 대신 콜론(:)
반복	<code>for(i=0; i<5; i++)</code>	<code>for i in range(5):</code>	Python은 범위(<code>range</code>) 중심
배열	<code>int a[3];</code>	<code>a = []</code>	C++은 크기가 고정됨
비어있음	<code>nullptr</code>	<code>None</code>	값이 없음을 나타내는 단어 차이
연결	<code>&&, , !</code>	<code>and, or, not</code>	C++은 기호, Python은 영어 단어
나눗셈	<code>/</code> (정수/실수 구분)	<code>/</code> (실수), <code>//</code> (정수)	Python은 정수 몫 기호가 따로 있음
주석	<code>//</code> 한 줄, <code>/*</code> 여러줄 <code>*/</code>	<code>#</code> 한 줄, <code>"""</code> 여러줄 <code>"""</code>	메모(주석)를 남기는 기호 차이
가져오기	<code>#include <...></code>	<code>import ..</code>	라이브러리를 불러오는 방식
클래스	<code>class A { };</code>	<code>class A:</code>	Python은 뒤에 콜론(:) 필수
접근제한	<code>private:</code> , <code>public:</code>	<code>_variable</code> (관례)	C++은 강제, Python은 약속
메모리	직접 관리 (<code>delete</code>)	자동 관리 (GC)	C++은 청소를 직접, Python은 자동